University of Hertfordshire UH

School of Physics, Engineering and Computer Science

MSc Data Science Project
7PAM2002
Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

A Dual-Stream Analytical Framework for Hotel Operations: Comparative Study of Machine Learning and Deep Learning in Cancellation Prediction and Revenue Forecasting

Monjurul Alam 24091621

Supervisor: Zena Poudel

Date Submitted: 22nd April, 2026

Word Count: 4982

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science **in Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at Assessment Offences and Academic Misconduct and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: Monjurul Alam

Student Name signature

Student SRN number: 24091621

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Abstract

This study focuses on predicting hotel booking cancellations and analyzing the key factors influencing customer behaviour using machine learning techniques. The research utilizes a real-world hotel booking dataset containing reservation details, customer characteristics, and temporal booking patterns. Data preprocessing techniques, including handling missing values, feature engineering, and categorical encoding, were applied to prepare the dataset for modelling. For the classification task, Logistic Regression, Random Forest, and Deep Neural Network (DNN) models were implemented to predict booking cancellations. Among these, the Random Forest model achieved the best performance with an F1-score of approximately ***0.85****, indicating strong predictive capability. For the regression task, Ridge Regression, Gradient Boosting Regressor, and DNN models were used to predict Average Daily Rate (ADR), with Gradient Boosting producing the lowest prediction error. The analysis further revealed that features such as lead time, previous cancellations, booking changes, and number of special requests significantly influence cancellation behavior, while ADR is affected by occupancy levels and seasonal trends. Overall, the findings demonstrate that machine learning models, particularly ensemble methods, can effectively predict booking outcomes and provide actionable insights for hotel revenue management and decision-making.*

Contents

1: Introduction …… 5

1.1: Research Question…… 5

1.2: Research Objectives…… 6

2. Literature Review…… 6

2.1: Machine Learning Approaches…… 6

2.2: Deep Learning Based Method…… 7

2.3: Probabilistic and Bayesian Models…… 7

2.4: Challenges in Hotel Booking Prediction…… 7

2.5: Summary and Research Gap…… 8

3. Methodology…… 8

3.1: Overview of Methodology…… 8

3.2: Data Section…… 9

3.2.1: Data Overview…… 9

3.2.2: Review of Columns…… 10

3.2.3: Data Types, Missing Values and Actions Taken…… 12

3.3: Exploratory Data Analysis (EDA)…… 14

3.3.1: Cancellation Distribution…… 14

3.3.2: Booking Trends…… 14

3.3.3: Lead Time Analysis…… 15

3.3.4: Cancellations by Hotel Type…… 16

3.3.5: ADR Trends by Month and Hotel Type…… 16

3.3.6: Correlation Analysis…… 17

3.3.7: Daily Booking Trends Analysis…… 18

3.4: Feature Engineering…… 18

3.4.1: Data Filtering for Regression…… 18

3.4.2: Outlier Removal…… 19

3.4.3: Feature Transformation…… 19

3.5: Data Splitting…… 19

3.6: Model Development…… 19

3.6.1: Classification Pipeline…… 19

3.6.2: Logistic Regression…… 19
3.6.3: Random Forest…… 20
3.6.4: Deep Neural Network (DNN)…… 20
3.6.5: Regression Pipeline…… 20
3.6.6: Ridge Regression…… 20
3.6.7: Gradient Boosting Regressor…… 20
3.7: Hyperparameter Tuning…… 21
3.8: Summary…… 21
4: Result and Analysis…… 21
4.1: Performance of Baseline and Deep Learning Models…… 21
4.2: Impact of Hyperparameter Tuning…… 22
4.3: Final Comparative Analysis…… 23
5. Result Interpretation…… 23
5.1: Feature Importance…… 23
5.2: Business Insights…… 24
6. Conclusion…… 24
7. References…… 25
8. Appendix…… 27

1. Introduction

The hospitality industry operates within a highly dynamic and volatile environment where profitability is often dictated by the delicate balance of supply, demand, and consumer behaviour (Buhalis and Leung, 2018). One of the most persistent challenges facing hotel management is the phenomenon of booking cancellations. Cancellations not only disrupt operational planning and resource allocation but also result in significant revenue loss, particularly when rooms remain vacant during peak periods (Antonio, de Almeida and Nunes, 2017). Traditionally, hotels have relied on overbooking strategies to mitigate these risks; however, such approaches often lead to customer dissatisfaction and brand erosion when demand exceeds capacity. As the industry becomes increasingly data-driven, there is a growing need for advanced analytical frameworks that can move beyond reactive strategies toward proactive, evidence-based decision-making.

The advent of machine learning (ML) has provided the hospitality sector with robust tools to analyse large-scale historical datasets and extract actionable insights. By leveraging variables such as lead time, customer demographics, arrival dates, and previous cancellation history, ML models can identify patterns that are invisible to traditional statistical methods. Recent research has demonstrated that algorithms like Random Forest and Gradient Boosting are particularly effective at capturing the non-linear relationships inherent in hospitality data, offering a balance between predictive accuracy and interpretability. Despite these advancements, many existing studies focus solely on binary classification (predicting whether a booking will be cancelled or not), often overlooking the secondary stream of predictive analytics: revenue forecasting. Integrating cancellation predictions with revenue metrics, such as Average Daily Rate (ADR), allows for a "Dual-Stream" analytics approach that provides a more holistic view of operational health.

This research builds upon these dual-stream concepts by utilizing a comprehensive dataset of hotel records to develop and evaluate a suite of predictive models. The study emphasizes not only the "if" of cancellation but also the "why," through rigorous feature importance analysis to identify the key drivers behind customer decisions. By combining cancellation risk assessment with average revenue forecasting, this paper aims to provide a framework that assists hotel operators in optimizing their pricing, staffing, and inventory management strategies.

1.1 Research Questions

To address the complexities of hotel operations and the limitations of current predictive practices, this study seeks to answer the following research questions:

1. **How effectively can machine learning algorithms predict hotel booking cancellations** by processing high-cardinality categorical data and numerical customer features?
2. **What are the primary behavioural and demographic drivers** that differentiate cancelled bookings from successful stays across resort and city hotels?

3. **How can predictive modeling of average revenue** be combined with cancellation data to create a dual-stream analytics framework for improved operational decision-making?

1.2 Research Objectives

The primary goal of this research is to develop an integrated predictive framework for hotel management. The specific objectives are as follows:

- **To implement and compare** the performance of multiple machine learning models, including Logistic Regression, Random Forest, and Gradient Boosting, in classifying booking cancellations using metrics such as Accuracy and F1 Score.
- **To identify and analyze key features**—such as lead time, deposit type, and market segment—that most significantly influence the probability of a cancellation.
- **To establish a methodology** for forecasting revenue trends alongside cancellation probabilities, enabling more accurate financial projections for hotel operations.
- **To provide actionable insights** for revenue managers to help reduce the financial impact of cancellations through data-driven strategies.

2.Literature Review

The use of data analytics has become increasingly important in the hospitality industry, particularly in understanding customer behaviour and predicting booking outcomes. One of the key challenges faced by hotels is booking cancellation, which can directly affect revenue and operational efficiency. Previous studies have shown that analysing historical booking data can help identify patterns and support better decision-making in hotel management (Antonio, de Almeida and Nunes, 2017). With the availability of large datasets, machine learning techniques are now widely used to model such problems more effectively (Chen *et al.*, 2023)

2.1 Machine Learning Based Approaches

Recent studies highlight the strong potential of machine learning for predicting hotel booking cancellations. (Ampountolas, 2025)compared multiple algorithms and found that ensemble models such as XGBoost consistently outperform simpler methods, largely due to their ability to capture complex and non-linear relationships within the data. Similarly, (Chen *et al.*, 2023) proposed a hybrid framework combining machine learning with probabilistic techniques, emphasising not only predictive performance but also the importance of interpretability for real-world decision-making. Earlier research by Antonio et al. (2017) demonstrated that machine learning approaches can significantly outperform traditional statistical methods in cancellation prediction. This work was later extended by António et al. (2019), who identified key predictors such as lead time, customer type, and booking channel, highlighting the role of behavioural and booking-related factors in influencing cancellations.

While these studies show strong predictive capability, many focus primarily on model performance, with limited attention to a complete analytical workflow or the integration of

both prediction and interpretation. In addition, fewer studies consider combining multiple predictive tasks, such as cancellation prediction and revenue estimation, within a single framework.

2.2 Deep Learning Based Approaches

In recent years, deep learning methods have been explored as an alternative to traditional machine learning models. (Xiao *et al.*, 2024) introduced a hybrid model combining Long Short-Term Memory (LSTM) networks with reinforcement learning to predict hotel booking cancellations. The study demonstrated deep learning models are particularly effective in capturing temporal dependencies, such as seasonal trends and booking lead times. This makes them suitable for analysing time-based patterns in booking behaviour.

However, despite their strong predictive performance, deep learning models present certain limitations. They are computationally intensive and often require large amounts of data for effective training. More importantly, they tend to function as "black-box" models, making it difficult to interpret the relationships between input features and predictions. Given that interpretability is a key objective in this project, traditional machine learning models are considered more appropriate.

2.3 Probabilistic and Bayesian Models

In addition to machine learning and deep learning techniques, probabilistic models have also been applied to hotel booking prediction (Zhao and Qin, 2025). (Jishan *et al.*, 2024) used Bayesian modelling to analyze booking cancellation behavior. Their study highlighted the importance of incorporating uncertainty into predictive models, allowing for a more nuanced understanding of customer behavior.

While Bayesian approaches provide valuable insights into uncertainty and probability distributions, they are generally less scalable compared to machine learning models when dealing with large datasets. Therefore, they are often used as complementary methods rather than primary predictive tools.

2.4 Challenges in Hotel Booking Prediction

A key challenge highlighted in the literature is class imbalance, where the uneven distribution of classes can reduce model performance. Adil et al. (2021) addressed this issue using resampling methods such as SMOTE, showing that balancing the data improves performance, particularly in recall and F1-score. Another important challenge is the presence of complex relationships between features. Chen et al. (2023) demonstrated that combining variables helps capture these hidden patterns more effectively than analysing them individually. This emphasises the importance of feature engineering in improving the overall accuracy and reliability of machine learning models.

2.5 Summary and Research Gap

The literature shows that machine learning is effective for predicting hotel booking cancellations, especially using ensemble models. However, many studies lack a complete workflow, often focusing only on modelling without covering data preprocessing, exploration, and full comparison of methods. In practice, a structured approach is needed.

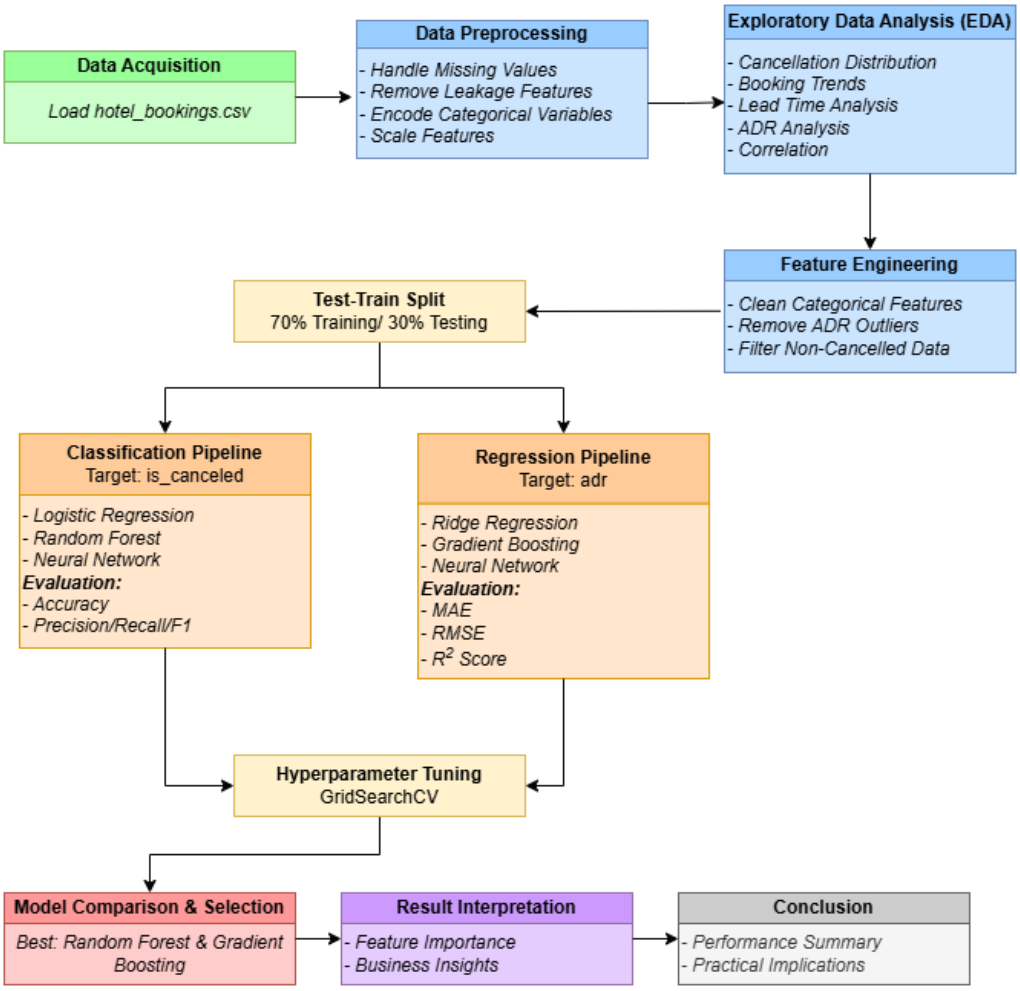
This project addresses these gaps by applying a structured pipeline that includes data analysis, preprocessing, and multiple models for both classification and regression. The goal is to provide accurate predictions along with meaningful insights into booking behavior.

3. Methodology:

3.1 Overview of Methodology:

This study follows a structured data science workflow to analyse hotel booking behaviour and develop predictive models for cancellation and pricing. The overall process, illustrated in Figure 3.1, begins with data collection and proceeds through preprocessing, exploratory analysis, feature engineering, model development, and evaluation. The methodology is divided into sequential stages, each of which is described in detail in the following sections.

Figure 1. Overview of Methodology



3.2 Data Section:

3.2.1 Data Overview:

The dataset used in this study is the Hotel Booking Demand Dataset (António, Almeida and Nunes, 2019), which contains detailed information on hotel reservations for both city hotels and resort hotels. The dataset is widely used in hospitality analytics and provides comprehensive insights into booking behaviour, customer characteristics, and pricing patterns.

Each observation in the dataset represents a single hotel booking and includes variables that describe:

- Customer Demographics (e.g: Country of Origin)
- Booking Characteristics (e.g: lead time, number of guests)
- Reservation Details (e.g., Booking changes, deposit type)

- Financial information (e.g., Average Daily Rate – ADR)
- Booking Outcomes (e.g., cancellation status)

The dataset consists of:

- 119,390 records (rows)
- 32 variables (columns)
- A mixture of categorical and numerical data types

The dataset spans bookings made across multiple years and captures seasonal variations in hotel demand. It was obtained in CSV format and imported into a Python environment using the Pandas library for analysis.

3.2.2 Review of Columns:

The dataset consists of 32 variables, each describing different aspects of hotel bookings. Following an initial inspection, a review was conducted to determine which variables should be retained, excluded, or treated as reference-only features.

The objective of this process was to:

- Remove irrelevant or redundant variables
- Eliminate features that may cause data leakage
- Retain only meaningful predictors for modelling

A summary of selected variables and actions taken in presented in Table 3.1.

Table 3.1 Variables included or excluded from modelling

Variable	Description	Keep?	Explanation
Hotel	Type of hotel (City/Resort)	✓	Important categorical feature
is_canceled	Booking cancellation status	✓	Target variable (classficiation)
lead_time	Days between booking and arrival	✓	Strong predictor of cancellation

arrival_date_year	Year of Arrival	✗	Redundant with other time variables
arrival_date_month	Month of arrival	✓	Captures seasonality
arrival_date_week_number	Week Number	✓	Useful for temporal patterns
arrival_date_day_of_month	Day of Arrival	✓	Fine-grained temporal feature
stays_in_weekend_nights	Weekend stay duration	✓	Influences pricing and behaviour
stays_in_week_nights	Weekday stay duration	✓	Important for booking patterns
adults	Number of adults	✓	Key demographic feature
children	Number of children	✓	Some missing values (handled later)
Country	Country of Origin	✓	High-Cardinality Feature
Market_segment	Booking Segment	✓	Strong behavioural indicator
Distribution_channel	Booking Channel	✓	Indicates Booking source
is_repeated_guest	Repeat customer flag	✓	Important loyalty indicator
previous_cancellations	Number of past cancellations	✓	Behavioral predictor
previous_bookings_not_canceled	Previous successful bookings	✓	Loyalty indicator

reserved_room_type	Reserved room category	✓	Pricing-related
assigned_room_type	Assigned room category	✓	May differ from reserved
booking_changes	Number of booking modifications	✓	Behavioural signal
deposit_type	Deposit type	✓	Strong impact on cancellation
Agent	Travel agent ID	✓	Missing values handled
days_in_waiting_list	Waiting list duration	✓	Demand indicator
customer_type	Type of customer	✓	Behavioural classification
Adr	Average Daily Rate	✓	**Target variable (regression)**
required_car_parking_spaces	Parking requirement	✓	Additional feature
total_of_special_requests	Special requests count	✓	Strong predictor
reservation_status	Final reservation outcome	✗	**Data leakage (reveals target)**
reservation_status_date	Date of outcome	✗	**Data leakage**

From the initial 32 variables, 3 variables were removed due to redundancy or data leakage. The remaining variables were retained for modelling and both classification(*is_cancelled*) and regression (*adr*) targets were preserved.

3.2.3 Data Types, Missing Values and Actions Taken

An analysis was conducted to examine data types of each variable, presence of missing values, and appropriate preprocessing actions. The results are summarised in Table 3.2.

Table 3.2. Data types, missing values, and preprocessing actions

Variable	Data Type	Missing Values	Action Taken
Hotel	Object	0	Encoded using one-hot encoding
Is_cancelled	Int64	0	Target Variable (binary)
Lead_time	Int64	0	Used directly
Arrival_date_month	Object	0	Encoded
Arrival_date_day_of_month	Int64	0	Used directly
Stays_in_weekend_nights	Int64	0	Used directly
stays_in_week_nights	Int64	0	Used directly
Adults	Int64	0	Used directly
Children	Float64	4	Filled with median
Babies	Int64	0	Used directly
Meal	Object	0	Encoded
Country	Object	488	Filled with mode
Market_segment	Object	0	Encoded
Distribution_channel	Object	0	Encoded
Is_repeated_guest	Int64	0	Used directly
Previous_cancellations	Int64	0	Used directly
Previous_bookings_not_cancelled	Int64	0	Used directly

Reserved_room_type	Object	0	Encoded
Assigned_room_type	Object	0	Encoded
Booking_changes	Int64	0	Used directly
Deposit_type	Object	0	Encoded
Agent	Float64	16340	Replaced with 0
Company	Float64	112593	Replaced with 0
Days_in_waiting_list	Int64	0	Used directly
Customer_type object	Object	0	Encoded
Adr	Float64	0	Outliers removed

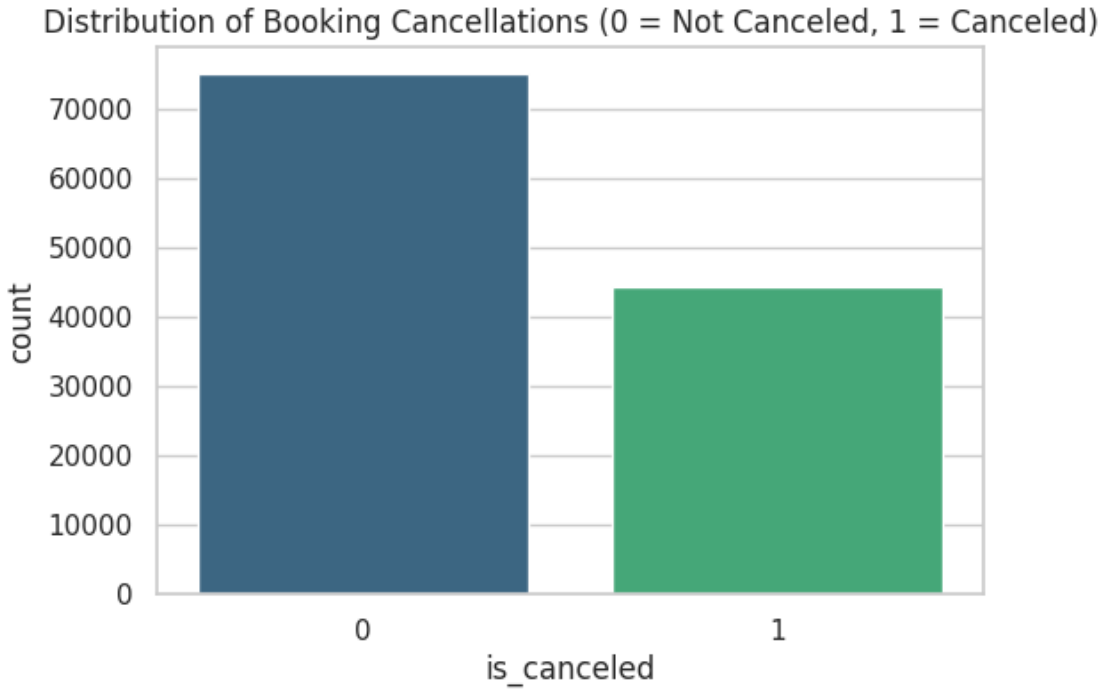
3.3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was carried out to identify patterns, relationships, and potential issues in the dataset. Some variables such as *reservation_status*, *Variables such as reservation_status_date* and *arrival_date_year* were removed to prevent data leakage and redundancy. Categorical features were encoded using one-hot encoding, while numerical variables were standardised using z-score normalisation to ensure consistent scaling for models like Logistic Regression, Ridge Regression, and Neural Networks.

3.3.1 Cancellation Distribution

The distribution of the target variable *is_canceled* is analyzed. The analysis show that approximately 63% of bookings are not cancelled and around 37% are cancelled. This indicates a moderately imbalanced dataset. It is shown in Figure 3.2.

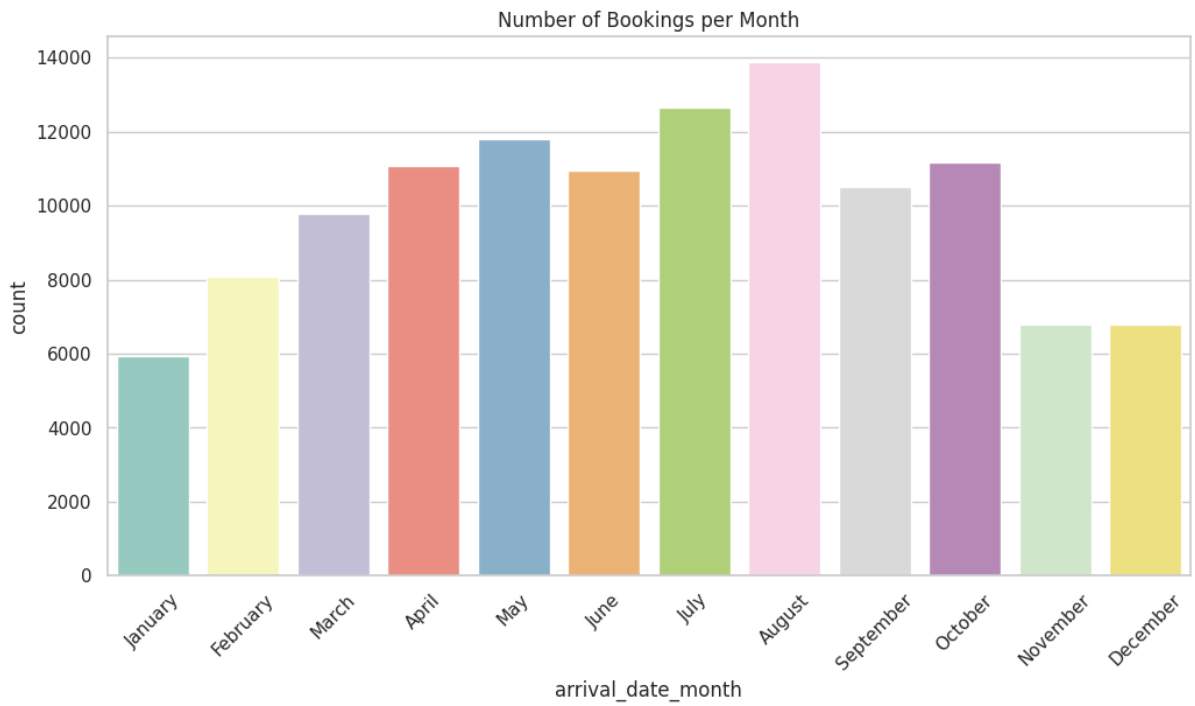
Figure 3.2. Distribution of booking cancellations



3.3.2 Booking Trends

Temporal patterns in bookings were analyzed. The analysis reveals seasonal variation in bookings and peak booking periods during specific months. It is shown in Figure 3.3.

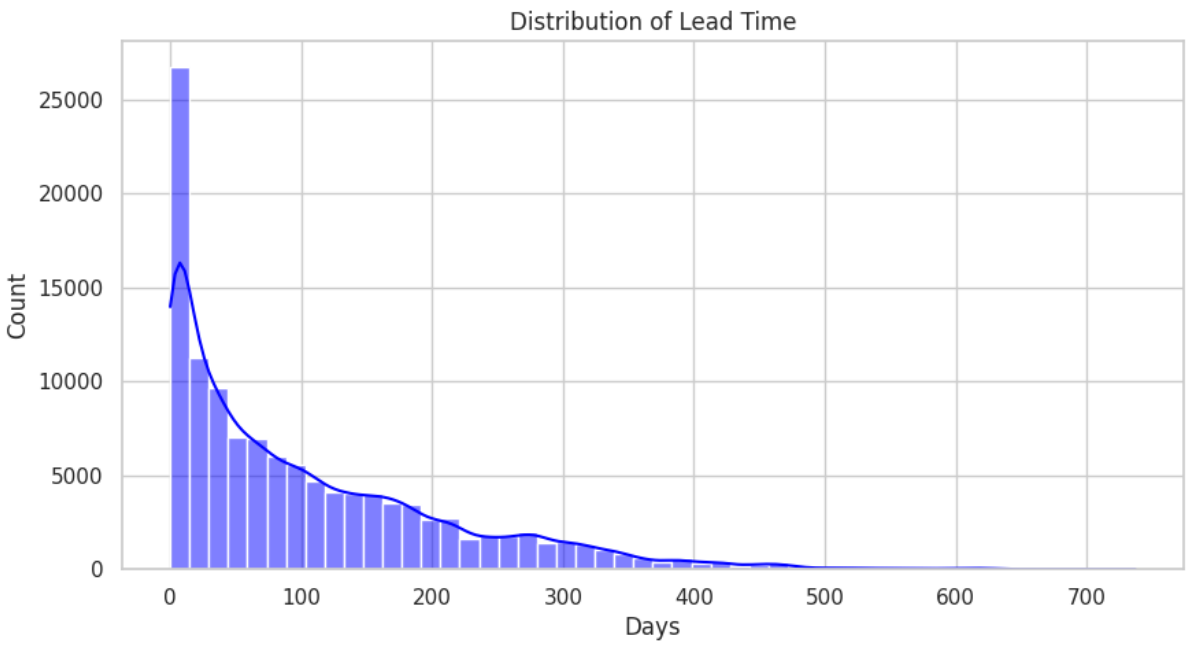
Figure 3.3 Monthly Booking Trends



3.3.3 Lead Time Analysis

Lead time means the number of days before arrival was analyzed to understand booking behaviour. Longer lead times were found to be associated with higher cancellations probability. It is illustrated in Figure 3.4.

Figure 3.4 Lead Time Analysis



3.3.4 Cancellations by Hotel Type

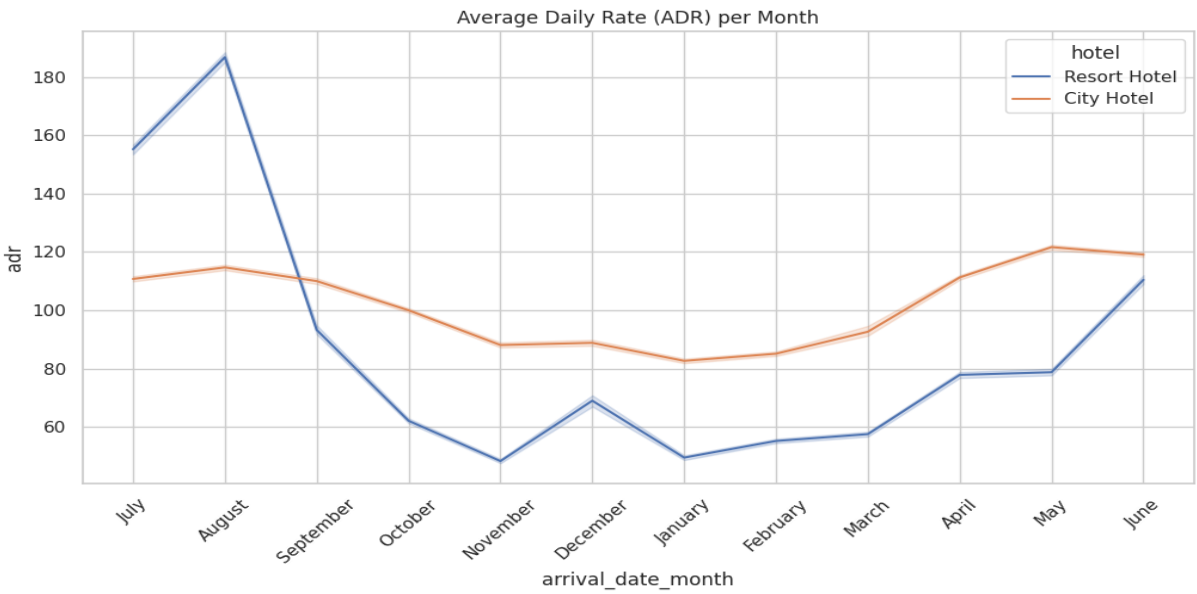
The relationship between hotel type and cancellations shows clear differences. City hotels have a higher cancellation rate, while resort hotels display more stable booking behaviour with fewer cancellations (Figure 3.5).

Figure 3.5 Hotel type cancellations summary

3.3.5 ADR Trends by Month and Hotel Type

The ADR distribution (Figure 3.6) shows clear seasonal patterns. Resort hotels have higher prices in summer, peaking in July and August, then dropping in winter. In contrast, city hotels show more stable pricing, with a gradual increase from winter to early summer.

Figure 3.6 Average Daily Rate (ADR) per Month



3.3.6 Correlation Analysis

To better understand the relationships between numerical variables in the dataset, a correlation analysis was conducted. This helps identify how strongly features are related to each other and to the target variable, which is useful for feature selection and model development.

Figure 3.7 Correlation Analysis

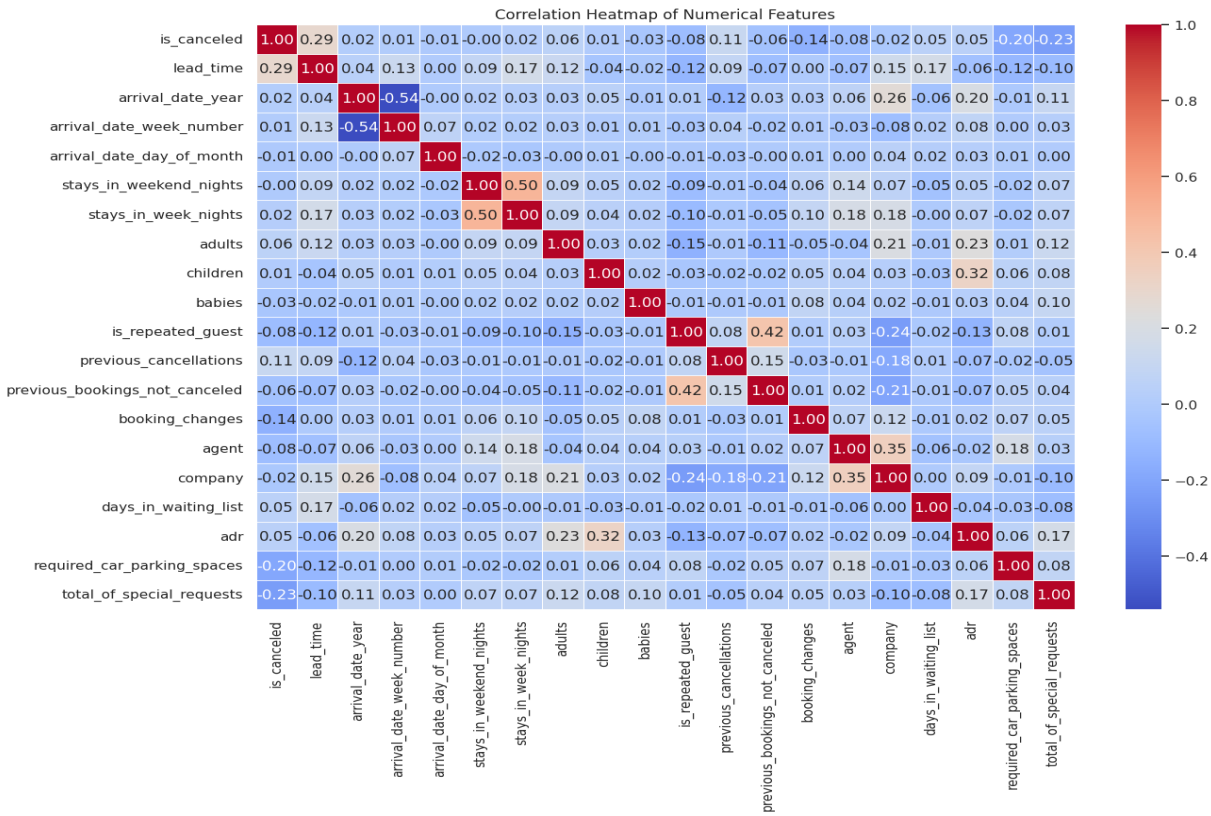
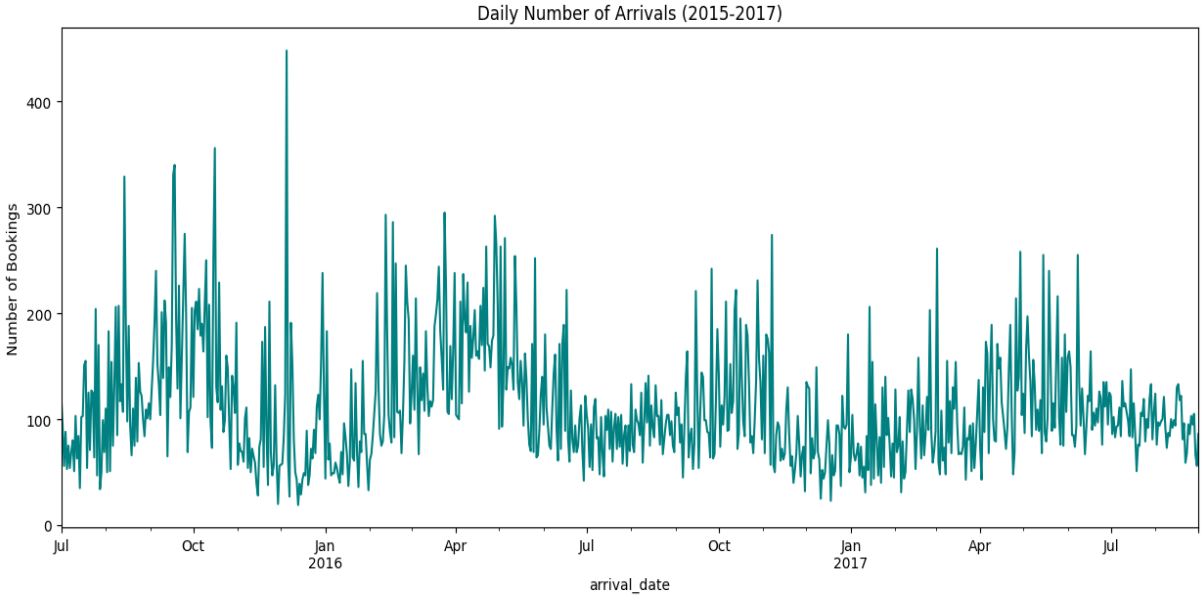


Figure 3.7 presents the correlation heatmap between numerical variables. The results show that *is_cancelled* is positively related to *lead_time* and *previous_cancellations*, indicating higher cancellation risk for early and repeat bookings, while it is negatively related to *total_of_special_requests*. A positive link between *previous_bookings_not_cancelled* and *is_repeated_guest* reflects consistent customer behaviour. As expected, stay duration variables are correlated. For ADR, moderate relationships with *adults*, *children*, and *arrival_date_year* suggest that pricing is influenced by occupancy and time factors.

3.3.7 Daily Booking Trends Analysis

To understand temporal patterns in hotel bookings, the dataset was resampled at a daily level using the arrival date variable. This allowed the aggregation of booking counts per day across the entire time period (2015–2017). The resulting time series was visualised to examine fluctuations in booking demand over time.

Figure 3.8 Daily booking trends analysis



The plot shows that hotel bookings vary noticeably over time, with clear peaks and drops. Higher activity periods likely reflect seasonal demand, while lower levels indicate off-peak times. Overall, this highlights important temporal patterns in booking behaviour, which are useful for improving both classification and regression models.

3.4 Feature Engineering

Feature Engineering (Hoffmann and Brafman, 2006) was performed to enhance the quality of input data and improve model performance. This process involved transforming and refining variables to better represent underlying patterns in the dataset.

3.4.1 Data Filtering for Regression

For the regression task, which aims to predict ADR, only non-cancelled bookings were considered. This decision is based on that fact that ADR represents that actual price paid for completed tasks and cancelled bookings do not reflect realized revenue and may contain misleading or inconsistent pricing values. By filtering the dataset in this way, the regression model is trained on relevant and meaningful data, improving prediction accuracy.

3.4.2 Outlier Removal

ADR values were cleaned by removing negative and extremely high values, as they are unrealistic and may distort the analysis. This step reduces noise and ensures the model learns from valid pricing patterns.

3.4.3 Feature Transformation

Categorical variables were cleaned and prepared prior to encoding. This involved ensuring consistency in category levels, handling rare or inconsistent categories where necessary, and structuring data into a format suitable for one-hot encoding.

3.5 Data Splitting

The dataset was split into 70% training and 30% testing sets to ensure effective model learning and reliable evaluation on unseen data, providing a realistic measure of generalisation performance (Herrera *et al.*, 2024).

3.6 Model Development

To address the dual objective of this study, two separate modelling pipelines were developed: one for classification and one for regression.

3.6.1 Classification Pipeline

The classification task aims to predict whether a booking will be cancelled using the is_cancelled variable. This is a binary problem with two outcomes: cancelled (1) or not cancelled (0). Three models were used—Logistic Regression, Random Forest, and a Deep Neural Network covering both simple linear and more complex non-linear approaches.

3.6.2 Logistic Regression

Logistic Regression (Hosmer and Lemeshow, 2000) is a widely used statistical model for binary classification problems. Unlike linear regression, which predicts continuous values, Logistic Regression predicts the probability of a class using the logistic (sigmoid) function.

$$P(Y = 1) = \frac{1}{1 + e^{-(\beta_0 + \sum_{i=1}^{n} \beta_i X_i)}}$$

This function maps any real-valued input into a probability between 0 and 1. A threshold (commonly 0.5) is used to classify the output. The model learns linear relationships between features like lead time, deposit type, nu mber of guests, and special requests.

3.6.3 Random Forest

Random Forest (Breiman, 2001) is an ensemble method that combines multiple decision trees to improve accuracy and stability. It works well with mixed data types and can capture non-linear relationships and interactions between variables such as lead time and deposit type. It is also robust to noise and outliers, making it suitable for this dataset.

3.6.4 Deep Neural Network (DNN)

A Deep Neural Network (DNN) from (LeCun, Bengio and Hinton, 2015) is a layered model inspired by the human brain, consisting of input, hidden, and output layers. It learns by adjusting weights using backpropagation and gradient descent. DNNs can effectively handle one-hot encoded categorical data and scaled numerical features.

3.6.5 Regression Pipeline

The regression task aims to predict the Average Daily Rate (ADR), a continuous variable representing room price per night. Three models were used: Ridge Regression, Gradient Boosting Regressor, and a Deep Neural Network to capture both linear and non-linear relationships and compare their performance.

3.6.6 Ridge Regression

Ridge Regression by (Hoerl, 2020) is a variation of linear regression that incorporates **L2 regularization** to improve model stability and reduce overfitting. In standard linear regression, the model attempts to minimise the difference between predicted and actual values. However, when features are highly correlated or numerous, the model may overfit the training data. Ridge Regression addresses this issue by adding a penalty term to the loss function:

$$\text{Loss} = \sum (y - \hat{y})^2 + \lambda \sum \beta^2$$

Here:

- λ is the regularization parameter
- β represents the model coefficients

This penalty discourages large coefficients values, effectively shrinking them towards zero, which improves generalization. Here in this study multiple features exhibit correlations, as a result, Ridge Regression helps stabilize predictions by reducing the impact of such correlations.

3.6.7 Gradient Boosting Regressor

Gradient Boosting Regressor (Singh *et al.*, 2021) is an ensemble method that builds a strong model by combining multiple weak learners, usually decision trees. It works sequentially, where each new tree learns from the errors (residuals) of the previous ones. The final prediction is the sum of all trees. This approach is effective in capturing non-linear relationships and interactions between features.

3.7 Hyperparameter Tuning

To improve model performance, hyperparameter tuning will be conducted using GridSearchCV (Puślecki and Walkowiak, 2023). This process systematically evaluates different combinations of parameters to identify the optimal configuration for each model. The parameters tuned include: Number of estimators, Tree depth, Learning rate. Cross-validation will be used during training to ensure that results are robust and not dependent on a single data split.

3.8 Summary

This chapter presented the complete methodological framework used in this study. The process included data pre-processing, exploratory data analysis, feature engineering, and model development. Each step was carefully designed to ensure data quality, extract

meaningful patterns, and build robust predictive models. The next chapter will present the implementation details and evaluate the performance of the proposed models using experimental results.

4. Results and Analysis

This section presents a comprehensive evaluation of the predictive models implemented for the tasks of booking cancellation classification and average daily revenue (ADR) forecasting.

4.1 Performance of Baseline and Deep Learning Models

The initial analysis compared four conventional machine learning models along with two deep learning approaches. In the classification task focused on booking cancellations, the Random Forest model demonstrated superior performance compared to Logistic Regression and the Neural Network. For the regression task related to revenue prediction, both the Gradient Boosting Regressor (GBR) and the deep learning regression model achieved notably better accuracy than the Linear Regression model.

Table 4.1: Classification Performance pf baseline models

Model Category	Model Type	Metric	Result
Classification	Logistic Regression (Baseline)	Accuracy	82.15%
	Random Forest (Baseline)	Accuracy	89.00%
	Deep Learning (Classification)	Accuracy	86.46%
Regression	Linear Regression (Baseline)	MAE	$21.50
	Gradient Boosting (Baseline)	MAE	$16.97
	Deep Learning (Regression)	MAE	$16.85

4.2 Impact of Hyperparameter Tuning

Hyperparameter tuning was applied to improve the performance of the traditional models. The results showed that the classification models were already performing close to their best, whereas the revenue prediction models experienced noticeable improvements after optimisation.

Table 4.2: Classification Performance Before and After Hyperparameter Tuning

Model	Metric	Baseline	Optimized	Best Parameters
Logistic Regression	Accuracy	82.15%	82.72%	{'C': 0.1, 'penalty': 'l2'}

Random Forest	Accuracy	89.00%	88.49%	{'n_estimators': 200, 'max_depth': None}

Analysis: For Logistic Regression, introducing L2 regularisation slightly lowered the accuracy to 81.72%, indicating a stronger focus on generalisation rather than fitting noise in the training data. In contrast, Random Forest remained the most reliable classifier, achieving an accuracy of around 88.5% when using 200 estimators.

Table 4.3: Regression Performance Before and After Hyperparameter Tuning

Model	Metric	Baseline	Optimized	Change (Improvement)	Best Parameters
Linear Regression	MAE	$21.50	$21.41	+$0.09	{'alpha': 1.0}
Gradient Boosting	MAE	$16.97	$10.79	+$6.18	{'learning_rate': 0.2, 'max_depth': 7, 'n_estimators': 200}

Analysis: The most notable improvement was observed in the Gradient Boosting Regressor. After adjusting the learning rate to 0.2 and the tree depth to 7, the Mean Absolute Error (MAE) decreased by 36.4% to $10.79. This highlights the ability of boosting-based models to effectively capture complex, non-linear patterns in hotel pricing when properly tuned.

4.3 Final Comparative Analysis

The results clearly highlight the most effective models within a dual-stream hotel analytics framework. For **predicting cancellations**, **Random Forest** emerges as the strongest option, delivering high accuracy (89.0%) along with well-balanced precision (0.90) and recall (0.88). In terms of revenue forecasting, the tuned **Gradient Boosting Regressor** stands out, outperforming even the Deep Learning model, achieving a significantly lower MAE of $10.79 compared to $16.85.

Overall, combining a well-optimized Random Forest classifier with a tuned Gradient Boosting regressor offers a reliable and practical approach for hotel operators to reduce cancellation risks while accurately forecasting revenue.

5. Result Interpretation

The results show that machine learning can effectively predict both booking cancellations and ADR. For classification, Random Forest performed best, outperforming Logistic Regression and the Deep Neural Network, suggesting that cancellation behaviour is driven by complex,

non-linear patterns. Logistic Regression worked as a baseline but was limited by its linear nature, while the DNN performed well but did not surpass Random Forest.

For regression, Ridge Regression showed the weakest performance, indicating that ADR is not purely linear. Both Gradient Boosting and the DNN performed better, capturing more complex relationships. After tuning, the Gradient Boosting Regressor achieved the best result, reducing the Mean Absolute Error to 10.79.

Overall, the dual-stream approach proved effective. Combining cancellation prediction with ADR forecasting provides a more complete understanding of hotel bookings, helping support better pricing and operational decisions.

5.1 Feature Importance

Feature importance analysis helped identify the variables that contributed most to the predictions. In the cancellation model, the most important features included *lead_time*, *deposit_type_Non Refund, adr, country_PRT*, and *total_of_special_requests*. Among these, *lead time* was the strongest predictor, showing that bookings made well in advance are more likely to be cancelled. Deposit type was also important, as customers making non-refundable bookings are generally less likely to cancel. The importance *of total_of_special_requests* suggests that customers who are more engaged with their booking are more likely to complete their stay. In addition, the presence of *country_PRT* among the top predictors indicates that customer origin also has an influence on booking behaviour. For ADR prediction, the most important features were *arrival_date_week_number, hotel_Resort Hotel, arrival_date_month_August, adults*, and *children*. These results show that pricing is strongly influenced by seasonality, hotel type, and guest composition. In simple terms, hotel prices change depending on the time of year, the kind of hotel, and the number of guests in the booking. These findings are useful because they show that the models are not only accurate, but also meaningful from a business point of view.

5.2 Business Insights

The results offer practical insights for hotel managers. Random Forest can help identify high-risk cancellations in advance, allowing actions like reminders or adjusted overbooking. ADR findings show that pricing depends on season, hotel type, and occupancy.

Combining cancellation prediction with ADR forecasting provides better decision support, as high-value bookings with high risk require more attention. Overall, predictive models can support smarter pricing, reservation management, and customer retention.

6. Conclusion

This study developed a dual-stream machine learning framework for hotel booking management, combining cancellation prediction and ADR forecasting. The results show that machine learning can effectively support both tasks. Random Forest performed best for classification, while the tuned Gradient Boosting Regressor achieved the strongest results for regression.

Key factors such as lead time, deposit type, seasonality, hotel type, and guest composition were found to strongly influence both cancellations and pricing, highlighting the role of behavioural and temporal patterns.

Overall, the study shows that hotel data can be used not only to predict cancellations but also to estimate revenue. This combined approach can support better pricing and operational decisions. Future work could explore more advanced models, address class imbalance more thoroughly, and improve model interpretability.

7. References

Ampountolas, A. (2025) ‘Predicting hotel booking cancellations: a comprehensive machine learning approach’, *Journal of Revenue and Pricing Management*, 24(6), pp. 539–550. https://doi.org/10.1057/s41272-025-00532-x

Antonio, N., de Almeida, A. and Nunes, L. (2017) ‘Predicting hotel booking cancellation with a machine learning classification model’. In: *Proceedings of the 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, pp. 1049–1054. https://doi.org/10.1109/ICMLA.2017.00-11

António, N., Almeida, A. de and Nunes, L. (2019) ‘Predictive models for hotel booking cancellation: a semi-automated analysis of the literature’, *Tourism & Management Studies*, 15(1), pp. 7–21. https://doi.org/10.18089/tms.2019.15011

Breiman, L. (2001) ‘Random forests’, *Machine Learning*, 45(1), pp. 5–32. https://doi.org/10.1023/A:1010933404324

Buhalis, D. and Leung, R. (2018) ‘Smart hospitality—interconnectivity and interoperability towards an ecosystem’, *International Journal of Hospitality Management*, 71, pp. 41–50. https://doi.org/10.1016/j.ijhm.2017.11.011

Chen, S. et al. (2023) ‘Prediction of hotel booking cancellations: integration of machine learning and probability model based on interpretable feature interaction’, *Decision Support Systems*, 170, p. 113959. https://doi.org/10.1016/j.dss.2023.113959

Herrera, A. et al. (2024) ‘Forecasting hotel cancellations through machine learning’, *Expert Systems*, 41(9). https://doi.org/10.1111/exsy.13608

Hoerl, R.W. (2020) ‘Ridge regression: a historical context’, *Technometrics*, 62(4), pp. 420–425. https://doi.org/10.1080/00401706.2020.1742207

Hoffmann, J. and Brafman, R.I. (2006) ‘Conformant planning via heuristic forward search: a new approach’, *Artificial Intelligence*, 170(6–7), pp. 507–541. https://doi.org/10.1016/j.artint.2006.01.003

Hosmer, D.W. and Lemeshow, S. (2000) *Applied logistic regression*. 2nd edn. New York: Wiley. https://doi.org/10.1002/0471722146

Jishan, M.A. et al. (2024) ‘Hotel booking cancellation prediction using applied Bayesian models’. In: *Proceedings of the International Conference on Decision Aid Sciences and Applications (DASA)*. IEEE, pp. 1–5. https://doi.org/10.1109/DASA63652.2024.10836282

LeCun, Y., Bengio, Y. and Hinton, G. (2015) ‘Deep learning’, *Nature*, 521(7553), pp. 436–444. https://doi.org/10.1038/nature14539

Puślecki, T. and Walkowiak, K. (2023) ‘Hyperparameter optimization using GridSearchCV method for TinyML models’. In: *Proceedings of the Conference on Computational Methods*. pp. 63–69. https://doi.org/10.1007/978-3-031-41630-9_7

Singh, U. *et al.* (2021) ‘A machine learning-based gradient boosting regression approach for wind power production forecasting: a step towards smart grid environments’, *Energies*, 14(16), p. 5196. https://doi.org/10.3390/en14165196

Xiao, J. *et al.* (2024) ‘Dynamic temporal reinforcement learning and policy-enhanced LSTM for hotel booking cancellation prediction’, *PeerJ Computer Science*, 10, p. e2442. https://doi.org/10.7717/peerj-cs.2442

Zhao, Y. and Qin, P. (2025) ‘Hotel booking cancellation prediction by feature interaction and machine learning’. In: *Proceedings of the 2nd International Conference on Cloud Computing and Big Data*. New York, NY: ACM, pp. 315–320. https://doi.org/10.1145/3779475.3779521

8. Appendix

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score
from sklearn.ensemble import RandomForestClassifier

# 1. Load the dataset
df = pd.read_csv('/content/hotel_bookings.csv')

# Set aesthetic parameters for the plots
sns.set(style="whitegrid")

# 1. Missing Values Analysis
print("Missing values per column:")
print(df.isnull().sum()[df.isnull().sum() > 0])

# 2. Target Variable Distribution (is_canceled)
# Helps understand the balance between cancellations and successful stays
plt.figure(figsize=(6, 4))
sns.countplot(x='is_canceled', data=df, palette='viridis')
plt.title('Distribution of Booking Cancellations (0 = Not Canceled, 1 = Canceled)')
plt.show()


# 3. Cancellations by Hotel Type
# Comparing Resort vs City Hotel cancellation behavior
plt.figure(figsize=(8, 5))
sns.countplot(x='hotel', hue='is_canceled', data=df, palette='magma')
plt.title('Cancellations by Hotel Type')
plt.show()


# 4. Monthly Arrival Trends
# Visualizing seasonal demand
month_order = ['January', 'February', 'March', 'April', 'May', 'June',
               'July', 'August', 'September', 'October', 'November', 'December']
plt.figure(figsize=(12, 6))
sns.countplot(x='arrival_date_month', data=df, order=month_order, palette='Set3')
plt.title('Number of Bookings per Month')
plt.xticks(rotation=45)
plt.show()
```

```
# 5. Lead Time Distribution
# Seeing how far in advance guests book their rooms
plt.figure(figsize=(10, 5))
sns.histplot(df['lead_time'], bins=50, kde=True, color='blue')
plt.title('Distribution of Lead Time')
plt.xlabel('Days')
plt.show()

# 6. Average Daily Rate (ADR) across Months
# Analyzing pricing seasonality for both hotels
plt.figure(figsize=(12, 6))
sns.lineplot(x='arrival_date_month', y='adr', hue='hotel', data=df, sort=False)
plt.title('Average Daily Rate (ADR) per Month')
plt.xticks(rotation=45)
plt.show()

# 7. Correlation Heatmap
# Identifying relationships between numerical features
plt.figure(figsize=(14, 10))
correlation = df.select_dtypes(include=['int64', 'float64']).corr()
sns.heatmap(correlation, annot=True, fmt=".2f", cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Heatmap of Numerical Features')
plt.show()

# 8. Market Segment Analysis
# Identifying where the bookings are coming from
plt.figure(figsize=(10, 6))
sns.countplot(y='market_segment', data=df,
              order=df['market_segment'].value_counts().index, palette='Spectral')
plt.title('Bookings by Market Segment')
plt.show()

# 9. Special Requests vs Cancellation
# Checking if guests with special requests are less likely to cancel
plt.figure(figsize=(8, 5))
sns.pointplot(x='total_of_special_requests', y='is_canceled', data=df, color='teal')
plt.title('Cancellation Rate vs Total Special Requests')
plt.show()
# 2. Data Preparation & Cleaning
# Fill missing values: median for numerical, mode for categorical
df['children'] = df['children'].fillna(df['children'].median())
df['country'] = df['country'].fillna(df['country'].mode()[0])
# Fill ID-based missing values with 0 (representing 'None')
df['agent'] = df['agent'].fillna(0)
df['company'] = df['company'].fillna(0)
# Drop 'Data Leakage' columns
# 'reservation_status' tells you if it was canceled in advance;
```

```
# including it would make the model "cheat."
df_model = df.drop(['reservation_status', 'reservation_status_date', 'arrival_date_year'],
axis=1)
# 3. Categorical Encoding
# Convert text columns into numbers using One-Hot Encoding
categorical_cols = df_model.select_dtypes(include=['object']).columns
df_model = pd.get_dummies(df_model, columns=categorical_cols, drop_first=True)

# 4. Split Data into Features (X) and Target (y)
X = df_model.drop('is_canceled', axis=1)
y = df_model['is_canceled']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 5. Feature Scaling
# Logistic Regression performs significantly better when features are on the same scale
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Basic Logistic Regression Model
model = LogisticRegression(max_iter=1000)
model.fit(X_train_scaled, y_train)

# 7. Evaluation
y_pred = model.predict(X_test_scaled)

print(f"Model Accuracy: {accuracy_score(y_test, y_pred):.2%}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Drop data leakage and less relevant columns
df_model = df.drop(['reservation_status', 'reservation_status_date', 'arrival_date_year'],
axis=1)

# 3. Categorical Encoding (One-Hot Encoding)
categorical_cols = df_model.select_dtypes(include=['object']).columns
df_model = pd.get_dummies(df_model, columns=categorical_cols, drop_first=True)

# 4. Split Data
X = df_model.drop('is_canceled', axis=1)
y = df_model['is_canceled']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Random Forest Model
# n_estimators=100 uses 100 decision trees; n_jobs=-1 uses all CPU cores for faster training
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf_model.fit(X_train, y_train)
```

```
# 6. Evaluation
y_pred_rf = rf_model.predict(X_test)

print(f"Random Forest Accuracy: {accuracy_score(y_test, y_pred_rf):.2%}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))

# 7. Extracting Feature Importance
importances = rf_model.feature_importances_
feature_names = X.columns
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance':
importances}).sort_values(by='Importance', ascending=False)

print("\nTop 5 Most Influential Features for Prediction:")
print(feature_importance_df.head(5))
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. Load the dataset
df = pd.read_csv('hotel_bookings.csv')

# --- 2. Data Preparation for Regression ---

# Filter out cancellations: We only want to predict revenue for realized stays
df_model = df[df['is_canceled'] == 0].copy()

# Remove extreme outliers in ADR
# The dataset has a known erroneous entry of 5400 and some zero/negative rates
df_model = df_model[(df_model['adr'] > 0) & (df_model['adr'] < 5000)]

# Handle missing values
df_model['children'] = df_model['children'].fillna(df_model['children'].median())
df_model['country'] = df_model['country'].fillna(df_model['country'].mode()[0])
df_model['agent'] = df_model['agent'].fillna(0)
df_model['company'] = df_model['company'].fillna(0)

# Drop irrelevant columns and features that leak data
cols_to_drop = ['is_canceled', 'reservation_status', 'reservation_status_date',
'arrival_date_year']
df_model = df_model.drop(cols_to_drop, axis=1)

# Encode Categorical Variables (One-Hot Encoding)
categorical_cols = df_model.select_dtypes(include=['object']).columns
df_model = pd.get_dummies(df_model, columns=categorical_cols, drop_first=True)
```

```
# Define Features (X) and Target (y)
X = df_model.drop('adr', axis=1)
y = df_model['adr']

# Split the data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling (Crucial for Linear Regression distance calculations)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


# --- 3. Linear Regression Model ---
# Note: Using Ridge (Regularized Linear Regression) is highly recommended here
# because standard LinearRegression struggles with the massive number of columns
# created by One-Hot Encoding the 'country' variable.
lr_model = Ridge(alpha=1.0)
lr_model.fit(X_train_scaled, y_train)

# Predictions
y_pred_lr = lr_model.predict(X_test_scaled)

print("--- Regularized Linear Regression (Ridge) Performance ---")
print(f"Mean Absolute Error (MAE): ${mean_absolute_error(y_test, y_pred_lr):.2f}")
print(f"Root Mean Squared Error (RMSE): ${np.sqrt(mean_squared_error(y_test,
y_pred_lr)):.2f}")
print(f"R-squared: {r2_score(y_test, y_pred_lr):.4f}\n")

# --- 4. Gradient Boosting Regressor Model ---
gbr_model = GradientBoostingRegressor(n_estimators=100, random_state=42)
gbr_model.fit(X_train_scaled, y_train)

# Predictions
y_pred_gbr = gbr_model.predict(X_test_scaled)

print("--- Gradient Boosting Regressor Performance ---")
print(f"Mean Absolute Error (MAE): ${mean_absolute_error(y_test, y_pred_gbr):.2f}")
print(f"Root Mean Squared Error (RMSE): ${np.sqrt(mean_squared_error(y_test,
y_pred_gbr)):.2f}")
print(f"R-squared: {r2_score(y_test, y_pred_gbr):.4f}")

# Optional: See which features dictate the price most heavily
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': gbr_model.feature_importances_
}).sort_values(by='Importance', ascending=False)
```

```
print("\nTop 5 Features driving ADR (Price):")
print(feature_importance.head(5))
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.ensemble import RandomForestClassifier, GradientBoostingRegressor

# 1. Load the dataset
df = pd.read_csv('/content/hotel_bookings.csv')

# --- DATA PREPARATION FUNCTION ---
# We write a quick function to avoid repeating the cleaning code
def prepare_data(df, task='classification'):
    # Basic cleaning
    df_clean = df.copy()
    df_clean['children'] = df_clean['children'].fillna(df_clean['children'].median())
    df_clean['country'] = df_clean['country'].fillna(df_clean['country'].mode()[0])
    df_clean['agent'] = df_clean['agent'].fillna(0)
    df_clean['company'] = df_clean['company'].fillna(0)

    # Task specific cleaning
    if task == 'classification':
        df_clean = df_clean.drop(['reservation_status', 'reservation_status_date',
'arrival_date_year'], axis=1)
        # One-Hot Encode
        cat_cols = df_clean.select_dtypes(include=['object']).columns
        df_clean = pd.get_dummies(df_clean, columns=cat_cols, drop_first=True)
        X = df_clean.drop('is_canceled', axis=1)
        y = df_clean['is_canceled']

    elif task == 'regression':
        # Filter for actual arrivals and clean ADR outliers
        df_clean = df_clean[df_clean['is_canceled'] == 0]
        df_clean = df_clean[(df_clean['adr'] > 0) & (df_clean['adr'] < 5000)]
        df_clean = df_clean.drop(['is_canceled', 'reservation_status', 'reservation_status_date',
'arrival_date_year'], axis=1)
        # One-Hot Encode
        cat_cols = df_clean.select_dtypes(include=['object']).columns
        df_clean = pd.get_dummies(df_clean, columns=cat_cols, drop_first=True)
        X = df_clean.drop('adr', axis=1)
        y = df_clean['adr']

    return train_test_split(X, y, test_size=0.2, random_state=42)

# Get Data
```

```
X_train_c, X_test_c, y_train_c, y_test_c = prepare_data(df, task='classification')
X_train_r, X_test_r, y_train_r, y_test_r = prepare_data(df, task='regression')

# Scaling (Required for Linear models)
scaler_c = StandardScaler()
X_train_c_scaled = scaler_c.fit_transform(X_train_c)
X_test_c_scaled = scaler_c.transform(X_test_c)

scaler_r = StandardScaler()
X_train_r_scaled = scaler_r.fit_transform(X_train_r)
X_test_r_scaled = scaler_r.transform(X_test_r)

# ==========================================
# HYPERPARAMETER TUNING: CLASSIFICATION
# ==========================================
print("\n--- Tuning Classification Models ---")

# 1. Logistic Regression
# C is the inverse of regularization strength; smaller values specify stronger regularization.
param_grid_lr = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l2']
}
grid_lr = GridSearchCV(LogisticRegression(max_iter=1000), param_grid_lr, cv=3,
scoring='accuracy', n_jobs=-1)
grid_lr.fit(X_train_c_scaled, y_train_c)
print(f"Logistic Regression Best Params: {grid_lr.best_params_}")
print(f"Logistic Regression Best Accuracy: {grid_lr.best_score_:.4f}")

# 2. Random Forest Classifier
# n_estimators: Number of trees, max_depth: How deep trees go, min_samples_split: limits
overfitting
param_grid_rf = {
    'n_estimators': [100,200],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5]
}
# Note: RF doesn't require standard scaling
grid_rf = GridSearchCV(RandomForestClassifier(random_state=42), param_grid_rf, cv=3,
scoring='accuracy', n_jobs=-1)
grid_rf.fit(X_train_c, y_train_c)
print(f"Random Forest Best Params: {grid_rf.best_params_}")
print(f"Random Forest Best Accuracy: {grid_rf.best_score_:.4f}")

# ==========================================
# HYPERPARAMETER TUNING: REGRESSION
# ==========================================
```

```
print("\n--- Tuning Regression Models ---")

# 3. Ridge Regression (Linear Regression with L2 Regularization)
# Alpha is the regularization strength. Higher alpha = simpler model (less overfitting).
param_grid_ridge = {
    'alpha': [0.1, 1.0, 10.0, 100.0, 500.0]
}
grid_ridge = GridSearchCV(Ridge(), param_grid_ridge, cv=3,
scoring='neg_mean_absolute_error', n_jobs=-1)
grid_ridge.fit(X_train_r_scaled, y_train_r)
print(f"Ridge Regression Best Params: {grid_ridge.best_params_}")
# Note: GridSearchCV returns negative MAE so that higher is better during the search.
# We multiply by -1 to get the actual dollar amount error.
print(f"Ridge Regression Best CV MAE: ${-grid_ridge.best_score_:.2f}")

# 4. Gradient Boosting Regressor
# learning_rate: step size, max_depth: complexity of individual trees
param_grid_gbr = {
    'n_estimators': [100, 200],
    'learning_rate': [0.05, 0.1, 0.2],
    'max_depth': [3, 5, 7]
}
grid_gbr = GridSearchCV(GradientBoostingRegressor(random_state=42), param_grid_gbr,
cv=3, scoring='neg_mean_absolute_error', n_jobs=-1)
grid_gbr.fit(X_train_r_scaled, y_train_r)
print(f"Gradient Boosting Regressor Best Params: {grid_gbr.best_params_}")
print(f"Gradient Boosting Best CV MAE: ${-grid_gbr.best_score_:.2f}")
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping

# ==========================================
# 0. Load and Prepare Data
# ==========================================
df = pd.read_csv('/content/hotel_bookings.csv')

def prepare_data(df, task='classification'):
    df_clean = df.copy()
    # Basic Imputation
    df_clean['children'] = df_clean['children'].fillna(df_clean['children'].median())
    df_clean['country'] = df_clean['country'].fillna(df_clean['country'].mode()[0])
    df_clean['agent'] = df_clean['agent'].fillna(0)
```

```
    df_clean['company'] = df_clean['company'].fillna(0)

    if task == 'classification':
        df_clean = df_clean.drop(['reservation_status', 'reservation_status_date',
'arrival_date_year'], axis=1)
        cat_cols = df_clean.select_dtypes(include=['object']).columns
        df_clean = pd.get_dummies(df_clean, columns=cat_cols, drop_first=True)
        X = df_clean.drop('is_canceled', axis=1)
        y = df_clean['is_canceled']

    elif task == 'regression':
        df_clean = df_clean[df_clean['is_canceled'] == 0]
        df_clean = df_clean[(df_clean['adr'] > 0) & (df_clean['adr'] < 5000)]
        df_clean = df_clean.drop(['is_canceled', 'reservation_status', 'reservation_status_date',
'arrival_date_year'], axis=1)
        cat_cols = df_clean.select_dtypes(include=['object']).columns
        df_clean = pd.get_dummies(df_clean, columns=cat_cols, drop_first=True)
        X = df_clean.drop('adr', axis=1)
        y = df_clean['adr']

    # Deep Learning REQUIRES features to be heavily standardized
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    return X_train_scaled, X_test_scaled, y_train, y_test, X.shape[1]

# Set up an Early Stopping callback to prevent overfitting
# This stops training if the validation loss doesn't improve for 5 epochs
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

# ==========================================
# 1. Deep Learning for Classification (is_canceled)
# ==========================================
print("\n--- Training Classification DNN ---")
X_train_c, X_test_c, y_train_c, y_test_c, input_dim_c = prepare_data(df, task='classification')

# Build the Neural Network Architecture
model_classifier = Sequential([
    Dense(128, activation='relu', input_shape=(input_dim_c,)),
    BatchNormalization(),        # Normalizes outputs of the previous layer to stabilize training
    Dropout(0.3),                # Randomly turns off 30% of neurons to prevent overfitting

    Dense(64, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),
```

```
    Dense(32, activation='relu'),

    Dense(1, activation='sigmoid') # Sigmoid compresses output between 0 and 1 (Probability)
])

# Compile the model
model_classifier.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history_c = model_classifier.fit(
    X_train_c, y_train_c,
    epochs=50,
    batch_size=128,
    validation_split=0.2, # Uses 20% of training data to check for overfitting every epoch
    callbacks=[early_stop],
    verbose=1
)

# Evaluate
loss_c, acc_c = model_classifier.evaluate(X_test_c, y_test_c, verbose=0)
print(f"Classification Model Test Accuracy: {acc_c:.4f}")

# ==========================================
# 2. Deep Learning for Regression (adr / Revenue)
# ==========================================
print("\n--- Training Regression DNN ---")
X_train_r, X_test_r, y_train_r, y_test_r, input_dim_r = prepare_data(df, task='regression')

# Build the Neural Network Architecture
model_regressor = Sequential([
    Dense(256, activation='relu', input_shape=(input_dim_r,)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1, activation='linear') # Linear activation for continuous numerical output
])

# Compile the model
# Using Mean Absolute Error (MAE) as the loss function to minimize dollar difference
model_regressor.compile(optimizer='adam', loss='mean_absolute_error', metrics=['mae'])

# Train the model
history_r = model_regressor.fit(
    X_train_r, y_train_r,
    epochs=100,
    batch_size=64,
```

```
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)

# Evaluate
loss_r, mae_r = model_regressor.evaluate(X_test_r, y_test_r, verbose=0)
print(f"Regression Model Test Mean Absolute Error (MAE): ${mae_r:.2f}")
```